Deployment, AWS, and Final Wrap-up (Days 84-90)

1. Production build (Day 84)

```
./mvnw clean package -DskipTests
java -jar target/Enterprise-Digital-Banking-System-0.0.1-SNAPSHOT.jar \
  --spring.profiles.active=prod
```

Confirm it starts cleanly with `prod` profile env vars set (`DB_PASSWORD` , `JWT_SECRET` , `DB_USERNAME` if not root).

2. AWS deployment (Day 85) — simplest viable path

For a portfolio project, **Elastic Beanstalk** is far less setup than raw EC2/ECS and still demonstrates real cloud deployment:

1. Push the built jar to Elastic Beanstalk (Java platform) via the AWS Console (Create Application → upload the jar directly) or the EB CLI (`eb init` , `eb create`).
2. Set environment variables in EB's console (Configuration → Software → Environment properties): `DB_PASSWORD` , `JWT_SECRET` , `SPRING_PROFILES_ACTIVE=prod` .
3. Database: use **RDS** (managed MySQL) rather than running MySQL yourself on the EC2 instance — set `SPRING_DATASOURCE_URL` to the RDS endpoint.
4. **Skip Kafka/Redis on AWS** for a first deployment unless you have time — MSK (managed Kafka) and ElastiCache (managed Redis) are genuinely more setup than this timeline likely allows. Deploy the banking app + RDS only; note Kafka/Redis as "containerized and documented, not yet deployed to managed cloud services" — an honest, defensible scope line for an interview.

Test: hit the EB-provided public URL's `/actuator/health` (if you did doc 03) or `/auth/register` from your own machine with curl — confirms it's actually reachable over the internet, not just locally.

3. GitHub cleanup (Day 86)

- `git status` — confirm nothing uncommitted is sitting around.
- Check `.gitignore` covers `target/` , `.env` , IDE files.
- README should state: tech stack, how to run locally (env vars needed, `docker compose up` for infra), and link to the two notification-service and banking-app repos from each other.

4. Documentation (Day 87)

You already have this covered more thoroughly than most portfolio projects: the **Banking System Notes** artifact (Entity → Repository → Service → Controller per module) plus the `docs/*.pdf` reference files. For Day 87 specifically: export or link the notes artifact from the README so a reviewer can find it without asking.

5. Resume (Day 88)

Bullet point structure that actually survives interview follow-up questions (claim something you can explain in depth, not something broad and vague):

> Built a Spring Boot banking backend (JWT auth, double-entry ledger, audit logging, Kafka-based event-driven notifications across two microservices) with Redis caching and Dockerized deployment to AWS.

Each clause above should map to something you can open and explain live if asked — that's the actual test, not the wording.

6. Interview prep (Day 89)

Questions worth having a tight answer ready for, since they go straight at design decisions actually made in this project:

- "Why two services instead of one?" → notification decoupling, Kafka reasoning (dead service can't fail a transaction).
- "Why does `Emi` reference `Loan` but not the reverse?" → avoids bidirectional JPA/N+1 trap, matches `Transaction` / `LedgerEntry`.
- "How do you prevent a stale cached balance?" → `@CacheEvict` on every write path, explain the risk if one's missed.
- "Walk me through what happens on a transfer" → account lookup → balance check → ledger double-entry → Kafka event → audit log. You can narrate this end to end from the Notes artifact.
- "What's the biggest bug you caught in review?" → the swapped approve/reject endpoints in `LoanController` — genuinely good story: invisible from reading one method, only caught by checking what each one actually called.

7. Final review (Day 90)

Run through: does the app still boot? (`./mvnw spring-boot:run`) Does `./mvnw test` pass? Is the README accurate to what's actually built right now, not what was planned? That's the whole checklist — resist adding new scope this late.